

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Subject Name: **DATABASE MANAGEMENT SYSTEMS**

Subject Code: **CS T53**

UNIT -V

Concurrency Control – Lock Based Protocols – Deadlock Handling – Multiple Granularity – Timestamp Based Protocols – Validation Based Protocols – Multiversion Schemes – Snapshot Isolation – Insert Operations, Delete Operations and Predicate Reads – Weak Levels of Consistency – Concurrency in Index Structures – Recovery – Failure Classification – Storage – Recovery and Atomicity – Recovery Algorithm – Buffer Management – Failure with Loss of Nonvolatile Storage – Early Lock Release and Logical Undo Operations.

Case Studies IBM DB2 Universal Database – My SQL.

1. What are the two types of locks? (**APR 2014**)(**NOV 2015**)

- Shared lock (0r) Term lock- if the transaction T_i has obtained the shared mode lock(denoted by S) on item Q then T_i can read but cannot write Q
- Exclusive lock- if the transaction T_i has obtained the exclusive mode lock(denoted by X) on item Q then T_i can both read and write Q.

2. What is deadlock? (**DEC 2018**)

The state where neither of these transaction can ever proceed with its normal execution. This situation is called deadlock.

3. What is locking protocol?

Each transaction in the system follows the set of rules called locking protocol, indicating when a transaction may lock and unlock each of the data item.

4. What are the two phases of the locking protocol?

- Growing phase- a transaction may obtain locks but may not release any lock.
- Shrinking phase- a transaction may release lock but may not obtain any new lock

5. What is timestamp based protocol?

The method of determining the serializability order is to select an ordering among transaction in advance. The common method for doing so is to use a timestamp ordering scheme.

6. What is timestamp?

With each transaction T_i in the system, we associate the unique fixed timestamp denoted by the $TS(T_i)$. This timestamp is assigned by the database system before the transaction T_i starts the execution.

7. What are the two timestamp values?

- **W-timestamp(Q)** – denotes the largest timestamp of any transaction that executed write(Q) successfully.
- **R-timestamp(Q)** – denotes the largest timestamp of any transaction that executed read(Q) successfully.

8. What is timestamp ordering protocol?

Timestamp ordering protocol ensures that any conflicting read and write operation are executed in the timestamp order.

9. What is Thomas' write protocol?

The modification of the timestamp ordering protocol is called thomas' write protocol. Suppose the transaction T_i issues write(Q)

- If $TS(T_i) < R\text{-timestamp}(Q)$ then the value of Q that T_i producing was previously needed and that it had been assumed that the value would never been produced. Hence the system rejects the write operation and rolls T_i back.
- If $TS(T_i) < W\text{-timestamp}(Q)$ then T_i is attempting to write an obsolete value of Q. hence this write operation can be ignored.
- Otherwise the system executes the write operation and sets $W\text{-timestamp}(Q)$ to $TS(T_i)$.

10. What are the three phases in validation based protocol? (MAY 2019)

- Read phase
- Validation phase
- Write phase

11. What are the three different timestamp in validation based protocol?

- Start(T_i)

➤ Validation(T_i)

➤ Finish(T_i)

12. What is the optimistic concurrency control? (MAY 2019)

The transaction executes optimistically assuming they will be able to finish execution and validate at the end. This validation scheme is called optimistic concurrency control

13. What the two principle methods in the deadlock problem?

➤ Deadlock prevention

➤ Deadlock detection and recovery

14. What are the two types of deadlock prevention scheme using timestamp?

➤ Wait – die scheme is a non preemptive technique.

➤ Wound-wait scheme is a preemptive scheme

15. What is starvation?

No transaction gets rolled back repeatedly and is never allowed to make progress.

16. What are the actions to be followed to recover from deadlock?

➤ Selection of a victim

➤ Rollback

➤ Starvation

17. What are the classifications of failure?

➤ Transaction failure

✓ Logical error

✓ System error

➤ System crash

➤ Disk failure

18. What is meant by Log Based Recovery? (NOV 2014)(NOV 2015)

- A log is kept on stable storage.
- The log is a sequence of log records, and maintains a record of update activities on the database.
- When transaction T_i starts, it registers itself by writing a $\langle T_i$

start>log record

- Before T_i executes write(X), a log record $\langle T_i, X, V1, V2 \rangle$ is written,
- where $V1$ is the value of X before the write, and $V2$ is the value to be written to x .

- Log record notes that *T_i* has performed a write on data item

19. What are the properties of transactions? (April 2015) (DEC 2018)

- **ACID property:** Atomicity, Consistency, Isolation and Durability

20. Distinguish between optimization and pessimistic locking (April 2015)

- In pessimistic locking, when a user opens a data to update it, a lock is granted. Other users can only view the data until the whole transaction of the data update is completed.
- In optimistic locking, a data is opened for updating by multiple users. A lock is granted only during the update transaction and not for the entire session. Due to this concurrency is increased and is a practical approach of updating the data.

21. What is PostgreSQL?

PostgreSQL (pronounced "post-gress-Q-L") is an open source relational database management system (DBMS) developed by a worldwide team of volunteers. PostgreSQL is not controlled by any corporation or other private entity and the source code is available free of charge.

22. What is Oracle?

This is a suite of tools for various aspects of application development, including tools for forms development, data modeling, reporting, and querying. The suite supports the UML standard for development modeling. Oracle Repository provides configuration management for database objects, forms applications, Java classes, ML files, and other types of files

23. What is IBM DB2 Universal Database?

DB2 has a long history and is considered by many to have been the first database product to use SQL (also developed by IBM) although Oracle released a *commercial* SQL database product somewhat earlier than IBM.

24. What is My SQL?

My SQL is a relational database management system (RDBMS), and ships with no GUI tools to administer My SQL databases or manage data contained within the databases. Users may use the included command line tools, or use My SQL "front-ends", desktop software and web applications that create and manage My SQL databases, build database structures, back up data, inspect status, and work with data records.

25. What is Microsoft SQL Server?

Microsoft SQL Server is a relational database management system developed by Microsoft. As a database, it is a software product whose primary function is to store and retrieve data as requested by other software applications, be it those on the same computer or those running on another computer across a network (including the Internet).

26.What is the database design tool used in Oracle?

The major database design tool used in Oracle is Oracle Designer, which translates business logic and data flow into a schema definitions and procedural scripts for application logic. Oracle Designer stores the design in Oracle Repository, which serves as a single point of metadata for the application.

27. What is a Warehouse Builder?

Warehouse Builder is a tool for design and deployment of all aspects of a data warehouse, including schema design, data mapping and transformations, data load processing, and metadata management. Oracle Warehouse Builder supports both 3NF and star schemas and can also import designs from Oracle Designer.

28. What is Oracle Discoverer?

Oracle Discoverer is a Web-based, ad-hoc query, reporting, analysis and Web publishing tool for end users and data analysts. It allows users to drill up and down on result sets, pivot data, and store calculations as reports that can be published in a variety of formats such as spreadsheets or HTML.

29. What is Oracle Express Server?

Oracle Express Server is a multidimensional database server. It supports a wide variety of analytical queries as well as forecasting, modeling and scenario management.

30. State the Object - Relational Features in Oracle. (APRIL 2014)

Oracle has extensive support for object – relational constructs, including:

- Object Types
- Collection Types
- Object Tables
- Table Functions
- Object Views
- Methods
- User – defined aggregate functions
- XML data types

31.What is a segment?

The space in a table is divided into units, called segments, that each contains data for a specific data structure. There are four types of segments.

1. Data Segments
2. Index Segments
3. Temporary Segments
4. Rollback Segments

32.What are the ways by which partitioning is provided by Oracle?

Oracle supports various kinds of horizontal partitioning of tables and indices, and this feature plays a major role in Oracles' ability to support very large databases. The several types of partitioning provided are:

1. Range Partitioning
2. Hash Partitioning
3. Composite Partitioning
4. List Partitioning

33.What are the various access methods provide by Oracle?

Data can be accessed through a variety of access method:

1. Full table scan.
2. Index scan.
3. Index fast full scan.
4. Index joins.
5. Cluster and hash cluster access.

34.State some of the major types of transformations and rewrites supported by Oracle. (NOV 2014)

Some of the major types of transformations and rewrites supported by Oracle are as follows:

1. View merging
2. Complex view merging
3. Sub query flattening
4. Materialized view rewrite
5. Star transformation.

35.What is SQL* Loader?

Oracle has a direct load utility, SQL* Loader, that supports fast parallel loads of large amounts of data from external files. It supports variety of data formats and it can perform various filtering operations on the data being loaded.

36.What is Free Space Control Record?

DB2 uses a space map record called *Free Space Control Record* (FSCR) to find free space in the table. The FCSR record usually contains a space map for 500 pages.

37.What are the various access methods supported by DB2?

DB2 supports a comprehensive set of access methods on relational tables, including:

1. Table scan
2. Index scan
3. Index-only scan
4. List pre fetch scan

5. Index and-ing scan
6. Index or-ing scan

38.State the various modes provided by DB2 for isolation.

For isolation, DB2 supports the repeatable read (RR), read stability (RS), cursor stability (CS), and uncommitted read (UR) modes.

39.What is DB2 Data Propagator?

DB2 Data Propagator is a product in the DB2 family that provides replication of data among DB2, other relational database systems such as Oracle, Microsoft SQL Server, Sybase SQL Server, and Informix, and non relational data sources.

40.State the various tools provided by DB2 for administration.

Additionally, DB2 supports tools such as:

7. Audit facility – maintaining audit trace of database actions.
8. Governor facility – controls the priority and execution times.
9. Query patroller facility – managing the query jobs in the system.
10. Trace and diagnostic facility for debugging.
11. Event monitoring facility – tracks the resources and events during system execution.

41.What are the uses of SQL Query Analyzer?

A database administrator or developer can use SQL Query Analyzer to:

1. Analyze queries.
2. Format SQL queries.
3. Use templates for stores procedures, functions, and basic SQL statements.
4. Drag object names from the Object Browser to the Query window.
5. Define personal keystroke and toolbar options.

42.What is SQL Profiler?

(APRIL 2014) (NOV 2014)

SQL Profiler is a graphical utility that allows database administrators to monitor and record database activity. SQL Profiler can display all server activity in real time, or it can create filters that focus on the actions of particular users, applications, or types of commands.

43.What are the various stages in compiling an SQL Statement?

An SQL statement is compiled as follows.

1. Parsing / binding.
2. Simplification / normalization.
3. Cost-based optimization.

4. Plan preparation.

44. How is logging implemented in SQL Server?

The transaction log records all changes made to the database and stores enough information to allow any changes to be undone (rolled back) or redone (rolled forward) in the event of a system failure or rollback request.

45. What are the different uses of memory within SQL Server process?

There are many different uses of memory within the SQL Server Process:

1. Buffer pool.
2. Dynamic memory allocation.
3. Plan and execution cache.
4. Large memory grants.

46. When we call a transaction is terminated?

(April 2011)

- As long as you don't **COMMIT** or **ROLLBACK** a transaction, it's still "running" and potentially holding locks.
- If your client (application or user) closes the connection the database, any still running transactions will be rolled back and terminated.

47. What are errors cause a transaction failure?

(April 2011)

- Transaction failure
 - ✓ Logical error
 - ✓ System error
- System crash
- Disk failure

48. What is the use pivot operator?

(April/May 2012)

PIVOT can be used to generate cross tabulation reports to summarize data as it creates a more easy understandable data in a user friendly format. PIVOT rotates a table-valued expression by turning the unique values from one column in the expression into multiple columns in the output i.e it rotates a rows to columns and aggregations where they are required on any remaining column values that are wanted in the final output.

49. Explain a shadow copy scheme

(April 2013)

It is simple, but efficient, scheme called the shadow copy schemes. It is based on making copies of the database called shadow copies that one transaction is active at a time. The scheme also assumes that the database is simply a file on disk.

50. Mention the two approaches to manage the Database buffer.

(April 2013) [DEC 2016]

A buffer is an 8-KB page in memory, the same size as a data or index page. Thus, the buffer cache is divided into 8-KB pages. The buffer manager manages the functions for reading data or index pages from the database disk files into the buffer cache and writing modified pages back to disk. A page remains in the buffer cache until the buffer manager needs the buffer area to read in more data. Data is written back to disk only if it is modified. Data in the buffer cache can be modified multiple times before being written back to disk. For more information, see Reading Pages and Writing Pages.

51.Name any four query languages

(April 2013)

- SQL
- MY SQL
- Oracle
- MS Access

52.What is locking protocol?

(Nov 2010)

A lock is a mechanism to control concurrent access to a data item. Data items can be locked in two modes :

1. exclusive (X) mode. Data item can be both read as well as written. X-lock is requested using lock-X instruction.
2. shared (S) mode. Data item can only be read. S-lock is requested using lock-S instruction. Lock requests are made to concurrency-control manager

54. List any 4 DB2 background process.

(April 2013)

db2agent - For user process
db2cart - for archive
db2dlock - dead lock detector
db2event - event handler
db2fcmdm - FCM daemon

55. Name any 4 query Languages.

(April 2013)

DQL[*Doctrine Query Language*],**PHQL**[*Phalcon Query Language*], **SQL**[*Structure Query Language*] and **XML Query Language**.

56. What are the features of DB2? (April 2015)

CPU/Performance Improvements
Virtual Storage Enhancements
Security Extensions
Improved Catalog Concurrency

57. State any four background process in Oracle? (April 2015)

Checkpoint Process (CKPT)

System Monitor Process (SMON)

Process Monitor Process (PMON)

Recoverer Process (RECO)

58. What is your understanding about atomicity?(NOV 2015)

Atomicity. In a transaction involving two or more discrete pieces of information, either all of the pieces are committed or none are.

59. Define WAL Protocol? [April 2017]

write-ahead logging (WAL) is a family of techniques for providing atomicity and durability (two of the ACID properties) in database systems. In a system using WAL, all modifications are written to a log before they are applied. Usually both redo and undo information is stored in the log.

60. Define Steal and no-force policies. [April 2017](April 2018)

Steal is the possibility of a buffer being stolen by a new transaction. Suppose transaction T1 updates Page7 and it is swapped with transaction T2. Now T2 is allocated a frame (holding modified Page7) by medium term scheduler.

A **no-force policy** is used in transaction control in database theory. The term **no-force** refers to the disk pages related to the actual database object being modified. With a **no-force policy**, when a transaction commits, the changes made to the actual objects are **not** required to be written to disk in-place (forced).

11 MARKS

1. Explain Microsoft SQL server architecture.

(April 2012) (Nov 2014)

Microsoft SQL Server is a relational database management system developed by Microsoft. As a database, it is a software product whose primary function is to store and retrieve data as requested by other software applications, be it those on the same computer or those running on another computer across a network (including the Internet). There are at least a dozen different editions of Microsoft SQL Server aimed at different audiences and for workloads ranging from small single-machine applications to large Internet-facing applications with many concurrent users. Its primary query languages are T-SQL and ANSI SQL.

Architecture

The protocol layer implements the external interface to SQL Server. All operations that can be invoked on SQL Server are communicated to it via a Microsoft-defined format, called Tabular Data Stream (TDS). TDS is an application layer protocol, used to transfer data between a database server and a client. Initially designed and developed by Sybase Inc. for their Sybase SQL Server relational database engine in 1984, and later by Microsoft in Microsoft SQL Server, TDS packets can be encased in other physical transport dependent protocols, including TCP/IP, Named pipes, and Shared memory. Consequently, access to SQL Server is available over these protocols. In addition, the SQL Server API is also exposed over web services.^[42]

2. Give an overview of IBM DB2 process and architecture in detail.

(April 2013) (Nov 2013)

[DEC 2016] (DEC2018)

DB2 has a long history and is considered by many^[who?] to have been the first database product to use SQL (also developed by IBM) although Oracle released a *commercial* SQL database product somewhat earlier than IBM did.

The name **DB2** was first given to the Database Management System or DBMS in 1983 when IBM released **DB2** on its MVS mainframe platform. Prior to this, a similar product was named SQL/DS on the VM platform. The earlier System 38 platform also contained a relational DBMS. System Relational, or System R, was a research prototype developed in the 1970s. **DB2** has its roots back to the beginning of the 1970s when E.F. Codd, working for IBM, described the theory of relational databases and in June 1970 published the model for data manipulation. To apply the model Codd needed a relational database language he named Alpha. At the time IBM didn't believe in the potential of Codd's ideas, leaving the implementation to a group of programmers not under Codd's supervision, who violated several fundamentals of Codd's relational model; the result was Structured English QUery Language or SEQUEL. When IBM released its first relational database product, they wanted to have a commercial-

quality sublanguage as well, so it overhauled SEQUEL and renamed the basically new language (System Query Language) SQL to differentiate it from SEQUEL.

When Informix acquired Illustra and made their database engine an object-SQL DBMS by introducing their Universal Server, both Oracle and IBM followed suit by changing their database engines to be capable of object-relational extensions. In 2001, IBM bought Informix and in the following years incorporated Informix technology into the DB2 product suite. Today, DB2 can technically be considered to be an object-SQL DBMS.

For some years DB2, as a full-function DBMS, was exclusively available on IBM mainframes. Later IBM brought DB2 to other platforms, including OS/2, UNIX and Windows servers, then Linux (including Linux on zSeries) and PDAs. This process occurred through the 1990s. The inspiration for the mainframe version of DB2's architecture came in part from IBM IMS, a hierarchical database, and its dedicated database manipulation language, IBM DL/I. DB2 is also embedded in the i5/OS operating system for IBM System i (iSeries, formerly the AS/400), and versions are available for z/VSE and z/VM.

An earlier version of the code that would become DB2 LUW (Linux, Unix, Windows) was part of an Extended Edition component of OS/2 called Database Manager. IBM extended the functionality of Database Manager a number of times, including the addition of distributed database functionality that allowed shared access to a database in a remote location on a LAN. Eventually IBM declared that insurmountable complexity existed in the Database Manager code, and took the difficult decision to completely rewrite the software in their Toronto Lab. The new version of Database Manager, called DB2 like its mainframe parent, ran on the OS/2 and RS/6000 platforms, was called DB2/2 and DB2/6000 respectively. Other versions of DB2, with different code bases, followed the same '/' naming convention and became DB2/400 (for the AS/400), DB2/VSE (for the DOS/VSE environment) and DB2/VM (for the VM operating system). IBM lawyers stopped this handy naming convention from being used and decided that all products needed to be called "product FOR platform" (for example, DB2 for OS/390). The next iteration of the mainframe and the server-based products were named DB2 Universal Database (or DB2 UDB), a name that had already been used for the Linux-Unix-Windows version, with the introduction of widespread confusion over which version (mainframe or server) of the DBMS was being referred to. At this point, the mainframe version of DB2 and the server version of DB2 were coded in entirely different languages (PL/S for the mainframe and C++ for the server), but shared similar functionality and used a common architecture for SQL optimization: the Starburst Optimizer.

Over the years DB2 has both exploited and driven numerous hardware enhancements, particularly on IBM System z with such features as Parallel Sysplex data sharing. In fact, DB2 UDB Version 8 for z/OS now requires a 64-bit system and cannot run on earlier processors, and DB2 for z/OS maintains certain unique software differences in order to serve its sophisticated customers.

Although the ultimate expression of software-hardware co-evolution is the IBM mainframe, to some extent that phenomenon occurs on other platforms as well, as IBM's software engineers collaborate with their hardware counterparts.

In the mid-1990s, IBM released a clustered DB2 implementation called DB2 Parallel Edition, which initially ran on AIX. This edition allowed scalability by providing a shared nothing architecture, in which a single large database is partitioned across multiple DB2 servers that communicate over a high-speed interconnect. This DB2 edition was eventually ported to all Linux, UNIX, and Windows (LUW) platforms and was renamed to DB2 Extended Enterprise Edition (EEE). IBM now refers to this product as the Database Partitioning Feature (DPF) and sells it as an add-on to their flagship DB2 Enterprise product.

In mid 2006, IBM announced "Viper," which is the codename for DB2 9 on both distributed platforms and z/OS. DB2 9 for z/OS was announced in early 2007. IBM claimed that the new DB2 was the first relational database to store XML "natively". Other enhancements include OLTP-related improvements for distributed platforms, business intelligence/data warehousing-related improvements for z/OS, more self-tuning and self-managing features, additional 64-bit exploitation (especially for virtual storage on z/OS), stored procedure performance enhancements for z/OS, and continued convergence of the SQL vocabularies between z/OS and distributed platforms.

In October 2007, IBM announced "Viper 2," which is the codename for DB2 9.5 on the distributed platforms. There were three key themes for the release, Simplified Management, Business Critical Reliability and Agile XML development.

In June 2009, IBM announced "Cobra" (the codename for DB2 9.7 for LUW). DB2 9.7 adds data compression for database indexes, temporary tables, and large objects. DB2 9.7 also supports native XML data in hash partitioning (database partitioning), range partitioning (table partitioning), and multi-dimensional clustering. These native XML features allows users to directly work with XML in data warehouse environments. DB2 9.7 also adds several features that make it easier for Oracle Database users to work with DB2. These include support for the most commonly-used SQL syntax, PL/SQL syntax, scripting syntax, and data types from Oracle Database. DB2 9.7 also enhanced its concurrency model to exhibit behavior that is familiar to users of Oracle Database and Microsoft SQL Server.

In October 2009, IBM introduced its second major release of the year when it announced DB2 pureScale. DB2 pureScale is a database cluster solution for non-mainframe platforms, suitable for Online Transaction Processing (OLTP) workloads. IBM based the design of DB2 pureScale on the Parallel Sysplex implementation of DB2 data sharing on the mainframe. DB2 pureScale provides a fault-tolerant architecture and shared-disk storage. A DB2 pureScale system can grow to 128 database servers, and provides continuous availability and automatic load balancing.

In 2009, it was announced that DB2 can be an engine in MySQL. This allows users on the System i platform to natively access the DB2 under the IBM i operating system (formerly called OS/400), and for users on other platforms to access these files through the MySQL interface. On the System i and its predecessors the AS/400 and the System/38, DB2 is tightly integrated into the operating system, and comes as part of the operating system. It provides journaling, triggers and other features.

In October 2010, IBM announced the general availability (GA) of DB2 10 for z/OS. DB2® 10 for z/OS® expands the value delivered to businesses by IBM's industry-leading mainframe data server through innovations in key areas:

- Improved operational efficiencies for "out-of-the-box" DB2 CPU savings
- Unsurpassed resiliency for business-critical information
- Rapid application and warehouse deployment for business growth
- Enhanced business analytics and data visualization solutions with QMF

Selected features that deliver these valuable benefits to any business include:

- When compared to running on DB2 9, depending on the workload, customers may experience reduced CPU utilization
- When compared to running DB2 9, up to five to ten times more concurrent users on a single subsystem by avoiding memory constraints
- Greater concurrency for data management, data definition, and data access, including DDL, BIND, REBIND, PREPARE, utilities, and SQL
- Additional online changes for data definitions, utilities, and subsystems
- Improved security with better granularity for administrative privileges, data masking, and audit capabilities
- Temporal or versioned data to understand system and business times at the database level (Bi-temporal feature is not available on Oracle or any other competing RDBMS products)
- pureXML™ and SQL enhancements to simplify portability from other database solutions
- Productivity improved for database administrators, application programmers, and systems administrators
- QMF Classic Edition, an optional for-charge feature, providing greater interoperability with other programs plus improved queries, forms, diagnostics, performance, and resource control

3. Explain various model of locking a data item. Also explain two-phases protocol.(NOV 2015)

Locking protocol:

A set of rules followed by all transactions while requesting and releasing locks. Locking protocols restrict the set of possible schedules. Ensure serializable schedules by delaying transactions that might violate serializability.

Lock-compatibility matrix tells whether two locks are compatible or not. Any number of transactions can hold shared locks on a data item. If any transaction holds an exclusive lock on a data item no other transaction may hold any lock on that item.

Locking Rules/Protocol:

- A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions.
- If a lock cannot be granted, the requesting transaction is made to wait till all incompatible locks held by other transactions have been released. The lock is then granted.

Lock 2

Lock 1

Pitfalls of Lock-Based Protocols:

- Too early unlocking can lead to non-serializable schedules. Too late unlocking can lead to deadlocks.

Example

Transaction T1 transfers \$50 from account B to account A. Transaction T2 displays the total amount of money in accounts A and B, that is, the sum of A + B.

Early unlocking can cause incorrect results, non-serializable schedules. If A and B get updated in-between the read of A and B, the displayed sum would be wrong.

e.g., A = \$100, B = \$200, display A + B shows \$250

1. X-lock(B)
2. read B
3. B := B-50
4. write B
5. U-lock(B)
6. S-lock(A)
7. read A
8. U-lock(A)
9. S-lock(B)
10. read B

11. U-lock(B)
12. display A + B
13. X-lock(A)
14. read A
15. A := A+50
16. write A
17. U-lock(A)

T1 T2

Late unlocking causes deadlocks. Neither $T1$ nor $T2$ can make progress: executing lock-S(B) causes $T2$ to wait for $T1$ to release its lock on B . executing lock-X(A) causes $T1$ to wait for $T2$ to release its lock on A . To handle a deadlock

one of $T1$ or $T2$ must be rolled back and its locks released.

1. X-lock(B)
2. read(B)
3. B := B-50
4. write(B)
5. S-lock(A)
6. read(A)
7. S-lock(B)
8. X-lock(A)

T1 T2

Two-Phase Locking Protocol:

A locking protocol that ensures conflict-serializable schedules. It works in two phases:

- *Phase 1: Growing Phase: transaction may obtain locks transaction may not release locks*
- *Phase 2: Shrinking Phase: transaction may release locks transaction may not obtain locks*
- When the first lock is released, the transaction moves from phase 1 to phase 2.
- Properties of the Two-Phase Locking Protocol. Ensures serializability It can be shown that the transactions can be serialized in the order of their lock points (i.e. the point where a transaction acquired its final lock).
- Does not ensure freedom from deadlocks. Cascading roll-back is possible. Modifications of the two-phase locking protocol

Strict two-phase locking

- * A transaction must hold all its exclusive locks till it commits/aborts
- * Avoids cascading roll-back

Rigorous two-phase locking

- All locks are held till commit/abort. Transactions can be serialized in the order in which they commit.
- Refine the two-phase locking protocol with lock conversions

Phase 1: Acquire a lock-S on item can acquire a lock-X on item can convert a lock-S to a lock-X (upgrade)

Phase 2: can release a lock-S, can release a lock-X, can convert a lock-X to a lock-S (downgrade)

* Ensures serializability; but still relies on the programmer to insert the various locking instructions.

* Strict and rigorous two-phase locking (with lock conversions) are used extensively in DBMS.

Automatic Acquisition of Locks:

- A transaction T_i issues the standard read/write instruction without explicit locking calls (by the programmer).
- The operation $\text{read}(D)$ is processed as:
if T_i has a lock on D then
 $\text{read}(D)$
else
 if necessary wait until no other transaction has a lock-X on D ;
 grant T_i a lock-S on D ;
 $\text{read}(D)$;
end
- $\text{write}(D)$ is processed as:
if T_i has a lock-X on D then
 $\text{write}(D)$
else
 if necessary wait until no other transaction has any lock on D ;
 if T_i has a lock-S on D then
 upgrade lock on D to lock-X
 else
 grant T_i a lock-X on D ;
 end
 $\text{write}(D)$;
end

All locks are released after commit or abort

Implementation of Locking:

- A lock manager can be implemented as a separate process to which transactions send lock and unlock requests.
- The lock manager replies to a lock request by sending a lock grant message (or a message asking the transaction to roll back, in case of a deadlock).
- The requesting transaction waits until its request is answered.
- The lock manager maintains a data structure called a lock table to record granted locks and pending requests.

4. Discuss various type of failures that occur in a system.

Failure Classification

Various types of failure that may occur in a system

1. Transaction failure

a. Logical errors - The Transaction cannot complete due to some internal error condition

b. System errors

The database system must terminate an active transaction due to an error condition. (e.g., deadlock)

2. System crash:

A power failure or other hardware or software failure causes the system to crash.

Fail stop assumption

A non-volatile storage contents are assumed to not be corrupted by system crash. Database systems have numerous integrity checks to prevent corruption of disk data

3. Disk failure

A head crash or similar disk failure destroys all or part of Disk. Destruction is assumed to be detectable: disk drives use checksums to detect failures

5. Write about Microsoft SQL server in detail

Microsoft SQL Server is a relational database management system developed by Microsoft. As a database, it is a software product whose primary function is to store and retrieve data as requested by other software applications, be it those on the same computer or those running on another computer across a network (including the Internet). There are at least a dozen different editions of Microsoft SQL Server aimed at different audiences and for workloads ranging from small single-machine applications to large Internet-facing applications with many concurrent users. Its primary query languages are T-SQL and ANSI SQL.

6. Explain Multidimensional Clustering in IBM.

Multidimensional clustering tables

Multidimensional clustering (MDC) provides an elegant method for clustering data in tables along multiple dimensions in a flexible, continuous, and automatic way. MDC can significantly improve query performance, in addition to significantly reducing the overhead of data maintenance operations, such as reorganization, and index maintenance operations during insert, update, and delete operations. MDC is primarily intended for data warehousing and large database environments, and it can also be used in online transaction processing (OLTP) environments.

Comparison of regular and MDC tables

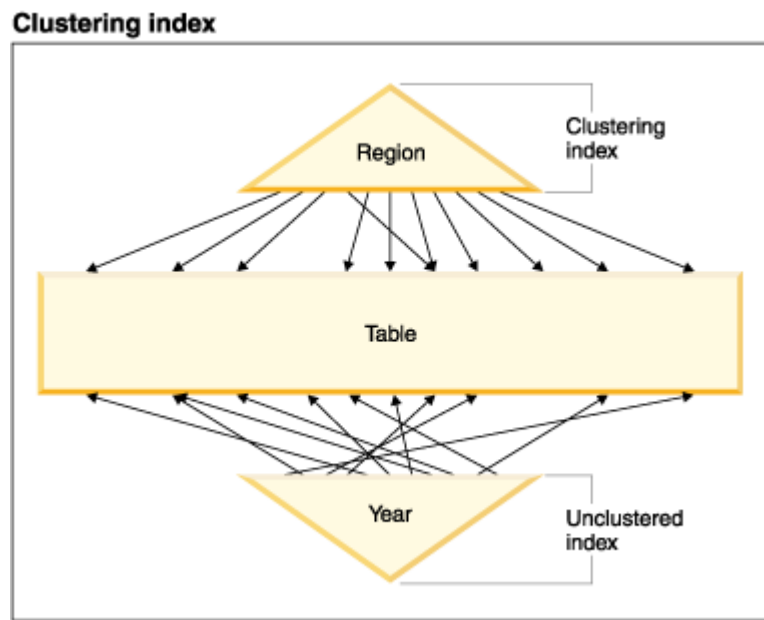
Regular tables have indexes that are record-based. Any clustering of the indexes is restricted to a single dimension. Prior to Version 8, DB2^(R) Universal Database (DB2 UDB) supported only single-dimensional clustering of data, through clustering indexes. Using a clustering index, DB2 UDB attempts to maintain the physical order of data on pages in the key order of the index when records are inserted and updated in the table. Clustering indexes greatly improve the performance of range queries that have predicates containing the key (or keys) of the clustering index. Performance is improved with a good clustering index because only a portion of the table needs to be accessed, and more efficient prefetching can be performed.

Data clustering using a clustering index has some drawbacks, however. First, because space is filled up on data pages over time, clustering is not guaranteed. An insert operation will attempt to add a record to a page nearby to those having the same or similar clustering key values, but if no space can be found in the ideal location, it will be inserted elsewhere in the table. Therefore, periodic table reorganizations may be necessary to re-cluster the table and to setup pages with additional free space to accommodate future clustered insert requests.

Second, only one index can be designated as the "clustering" index, and all other indexes will be unclustered, because the data can only be physically clustered along one dimension. This limitation is related to the fact that the clustering index is record-based, as all indexes have been prior to Version 8.1.

Third, because record-based indexes contain a pointer for every single record in the table, they can be very large in size.

Figure 26. A regular table with a clustering index



The table in Figure 26 has two record-based indexes defined on it:

- A clustering index on "Region"
- Another index on "Year"
-

The "Region" index is a clustering index which means that as keys are scanned in the index, the corresponding records should be found for the most part on the same or neighboring pages in the table. In contrast, the "Year" index is unclustered which means that as keys are scanned in that index, the corresponding records will likely be found on random pages throughout the table. Scans on the clustering index will exhibit better I/O performance and will benefit more from sequential prefetching, the more clustered the data is to that index.

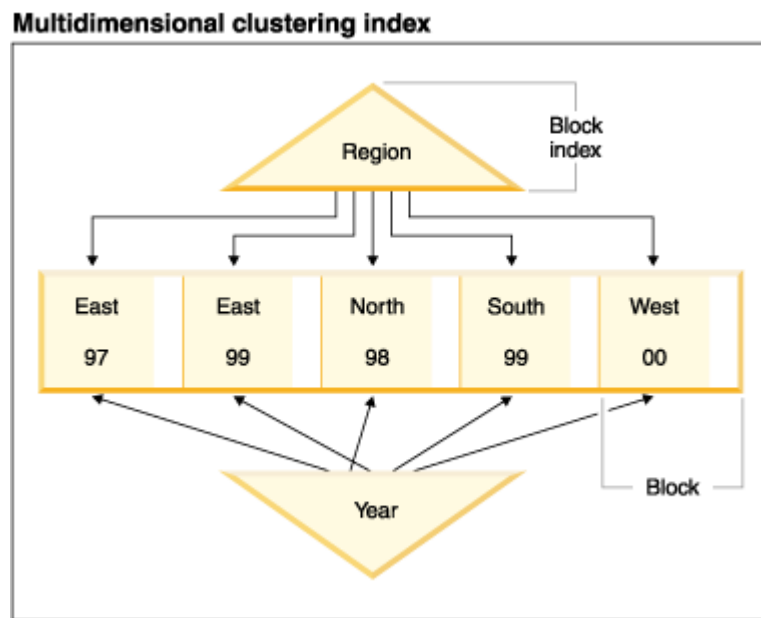
MDC introduces indexes that are block-based. "Block indexes" point to blocks or groups of records instead of to individual records. By physically organizing data in an MDC table into blocks according to clustering values, and then accessing these blocks using block indexes, MDC is able not only to address all of the drawbacks of clustering indexes, but to provide significant additional performance benefits.

First, MDC enables a table to be physically clustered on more than one key, or dimension, simultaneously. With MDC, the benefits of single-dimensional clustering are therefore extended to multiple dimensions, or clustering keys. Query performance is improved where there is clustering of one or more specified dimensions of a table. Not only will these queries access only those pages having records with the correct dimension values, these qualifying pages will be grouped into blocks, or extents.

Second, although a table with a clustering index can become unclustered over time, an MDC table is able to maintain and guarantee its clustering over all dimensions automatically and continuously. This eliminates the need to reorganize MDC tables to restore the physical order of the data.

Third, in MDC the clustering indexes are block-based. These indexes are drastically smaller than regular record-based indexes, so take up much less disk space and are faster to scan.

Figure 27. A multidimensional clustering table



Block indexes

The physically organized such that records having the same "Region" and "Year" values are grouped together into separate blocks, or extents. An extent is a set of contiguous pages on disk, so these groups of records are clustered on physically contiguous data pages. Each table page belongs to exactly one block, and all blocks are of equal size (that is, an equal number of pages). The size of a block is equal to the extent size of the table space, so that block boundaries line up with extent boundaries. In this case, two block indexes are created, one for the "Region" dimension, and another for the "Year" dimension. These block indexes contain pointers only to the blocks in the table. A scan of the "Region" block index for all records having "Region" equal to "East" will find two blocks that qualify. All records, and only those records, having "Region" equal to "East" will be found in these two blocks, and will be clustered on those two sets of contiguous pages or extents. At the same time, and completely independently, a scan of the "Year" index for records between 1999 and 2000 will find three blocks that qualify. A data scan of each of these three blocks will return all records and only those records that are between 1999 and 2000, and will find these records clustered on the sequential pages within each of the blocks.

In addition to these clustering improvements, MDC tables provide the following benefits:

- Probes and scans of block indexes are much faster due to their incredibly small size in relation to record-based indexes
- Block indexes and the corresponding organization of data allows for fine-grained "partition elimination", or selective table access
- Queries that utilize the block indexes benefit from the reduced index size, optimized prefetching of blocks, and guaranteed clustering of the corresponding data
- Reduced locking and predicate evaluation is possible for some queries
- Block indexes have much less overhead associated with them for logging and maintenance because they only need to be updated when adding the first record to a block, or removing the last record from a block
- Data rolled in can reuse the contiguous space left by data previously rolled out.

7. Explain various models of locking a data item. Also explain two-phase locking protocol.(April 2018)

Lock:

- Mechanism to control concurrent access to a data item.
- Lock requests are made to concurrency-control manager.
- Transaction can proceed only after request is granted. Data items can be locked in two modes:
 1. exclusive mode (X): Data item can be both read as well as written. X-lock is requested using lock-X(A) instruction.
 2. shared mode (S): Data item can only be read. S-lock is requested using lock-S(A) instruction. Locks can be released: U-lock(A)

Two-Phase Locking Protocol:

A locking protocol that ensures conflict-serializable schedules. It works in two phases:

- Phase 1: Growing Phase: transaction may obtain locks transaction may not release locks
- Phase 2: Shrinking Phase: transaction may release locks transaction may not obtain locks
- When the first lock is released, the transaction moves from phase 1 to phase 2.
- Properties of the Two-Phase Locking Protocol. Ensures serializability It can be shown that the transactions can be serialized in the order of their lock points (i.e. the point where a transaction acquired its final lock).
- Does not ensure freedom from deadlocks. Cascading roll-back is possible. Modifications of the two-phase locking protocol

Strict two-phase locking

- * A transaction must hold all its exclusive locks till it commits/aborts
- * Avoids cascading roll-back

Rigorous two-phase locking

- All locks are held till commit/abort. Transactions can be serialized in the order in which they commit.
- Refine the two-phase locking protocol with lock conversions

Phase 1: Acquire a lock-S on item can acquire a lock-X on item can convert a lock-S to a lock-X (upgrade)

Phase 2: can release a lock-S, can release a lock-X, can convert a lock-X to a lock-S (downgrade)

- * Ensures serializability; but still relies on the programmer to insert the various locking instructions.
- * Strict and rigorous two-phase locking (with lock conversions) are used extensively in DBMS.

Automatic Acquisition of Locks:

- A transaction T_i issues the standard read/write instruction without explicit locking calls (by the programmer).
- The operation $\text{read}(D)$ is processed as:
if T_i has a lock on D then
 $\text{read}(D)$
else
 if necessary wait until no other transaction has a lock-X on D ;
 grant T_i a lock-S on D ;
 $\text{read}(D)$;
end
- $\text{write}(D)$ is processed as:
if T_i has a lock-X on D then
 $\text{write}(D)$
else

 if necessary wait until no other transaction has any lock on D ;
 if T_i has a lock-S on D then
 upgrade lock on D to lock-X
 else
 grant T_i a lock-X on D ;
 end
 $\text{write}(D)$;
end

All locks are released after commit or abort.

Implementation of Locking:

- A lock manager can be implemented as a separate process to which transactions send lock and unlock requests.
- The lock manager replies to a lock request by sending a lock grant message (or a message asking the transaction to roll back, in case of a deadlock).
- The requesting transaction waits until its request is answered.
- The lock manager maintains a data structure called a lock table to record granted locks and pending requests.

Lock table:

- Implemented as in-memory hash table indexed on the data item being locked. Black rectangles indicate granted locks.
- White rectangles indicate waiting requests. Records also the type of lock granted/requested.
- Processing of requests:
 - New request is added to the end of the queue of requests for the data item, and granted if it is compatible with all earlier locks.
 - Unlock requests result in the request being deleted, and later requests are checked to see if they can now be granted.
 - If transaction aborts, all waiting or granted requests of the transaction are deleted. Index on transaction to implement this efficiently.

8. Explain deadlock handling in detail

(Nov 2012) (MAY 2019)

In order to understand deadlock, let us consider the following example:

T1	T2	Concurrency Controller
lock-X(A)		Lock granted
	lock-X(B)	Lock granted
lock-X(B)		Request is queued i.e T1 will wait for T2 to release the lock on data item B
	lock-X(A)	Request is again queued i.e T2 will wait for T1 to release the lock on data item A

As shown in partial schedule transaction T1 is waiting for transaction T2 to release its lock on data item B and transaction T2 is waiting for transaction T1 to release its lock on data item A. Such a cycle of transactions waiting for locks to be released is called a Deadlock.

Clearly, these two transactions will make no further progress. Now we can define the deadlock as "A system is in a deadlock state if there exists a set of transactions such that every transaction in the set is waiting for another transaction in the set."

More precisely, there exists a set of waiting transactions (T0, T1, Tn) such that T0 is waiting for a data item that is held by T1 and T1 is waiting for a data item that is held by T2 and

..... Tn-1 is waiting for a data item that is held by Tn and Tn is waiting for a data item that is held by T0.

None of the transactions can make progress in such a situation.

There are two principal methods for dealing with the deadlock problem.

- Deadlock prevention
- Deadlock detection

Deadlock prevention: We can use a deadlock-prevention protocol to ensure that the system will never enter a deadlock state.

Deadlock detection: In this case, we can allow the system to enter a deadlock state, and then try to recover using a deadlock detection and deadlock recovery scheme.

Both the above methods may result in transaction rollback. Prevention is commonly used if the probability that the system would enter a deadlock state is relatively high; otherwise detection and recovery are more efficient.

Deadlock Prevention

We can prevent deadlocks by giving each transaction a priority and ensuring that lower priority transactions are not allowed to wait for higher priority transactions (or vice versa). One way to assign priorities is to give each transaction a timestamp when it starts up. The lower the timestamp, the higher the transaction priority that is, the oldest transaction has the highest priority.

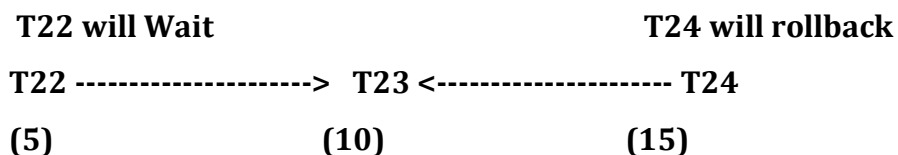
If a transaction T_i requests a lock and transaction T_j holds a conflicting lock, the lock manager can use one of the following two policies:

Wait-die

If T_i has higher priority, it is allowed to wait; otherwise it is aborted. It means when transaction T_i requests a data item currently held by T_j , T_i is allowed to wait only if it has a timestamp smaller than that of T_j (that is T_i is older than T_j). Otherwise T_i is rolled back (dies).

Example

Suppose that transactions T_{22} , T_{23} and T_{24} have timestamps 5, 10 and 15 respectively. If T_{22} requests a data item held by T_{23} then T_{22} will wait. If T_{24} requests a data item held by T_{23} then T_{24} will be rolled back.



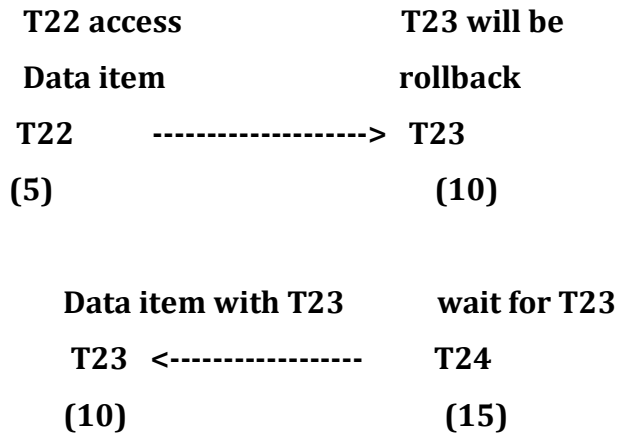
The wait-die scheme is non-preemptive scheme because only a transaction requesting a lock can be aborted. As a transaction grows older (and its priority increases), it tends to wait for more and more younger transactions.

Wound-wait

If T_i has higher priority, abort T_j otherwise T_i waits. It means when transaction T_i requests a data item currently held by T_j , T_i is allowed to wait only if it has a timestamp larger than that of T_j (that is T_i is younger than T_j). Otherwise T_j is rolled back (T_j is wounded by T_i).

Example:

Returning to our previous example, with transactions T22, T23 and T24, if T22 requests a data item held by T23 then the data item will be preempted from T23 and T23 will be rolled back. If T24 requests a data item held by T23, and then T24 will wait.



This scheme is based on a preemptive technique and is a counterpart to the wait-die scheme. In the wait-die scheme, lower priority transactions can never wait for higher priority transactions. In the wound-wait scheme, higher priority transactions never wait for lower priority transactions. In either case no deadlock cycle can develop.

When a transaction is aborted and restarted, it should be given the same timestamp that it had originally. Reissuing timestamps in this way ensures that each transaction will eventually become the oldest transaction, and thus the one with the highest priority, and will get the locks that it requires.

Problem of starvation

Whenever transactions are rolled back, it is important to ensure that there is no starvation that is, no transaction gets rolled back repeatedly and is never allowed to make progress.

Both the wound-wait and the wait-die schemes avoid starvation. At any time, there is a transaction with the smallest timestamp. This transaction cannot be required to roll back in either scheme. Since timestamps always increase, and since transactions are not assigned new timestamps when they are rolled back, a transaction that is rolled back will eventually have the smallest timestamp. Thus it will not be rolled back again.

Differences between wait-die and wound-wait scheme

There are following differences between wait-die and wound-wait schemes:

- In the wait-die scheme an older transaction must wait for a younger one to release its data item. Thus, the older the transaction gets the more it tends to wait. By contrast, in the wound wait scheme, an older transaction never waits for a younger transaction.
- In the wait-die scheme, if a transaction T24 request a data item held a data item by T23 and T24 then die and roll back because T_i request a data held by T_j and $TS(T_i) \gg TS(T_j)$. Now T24 restarted and

may reissue the same sequence of requests if data item is still held by T23 then T24 will die again. Thus T24 may die several times before acquiring the needed data item.

Now see what happens in case of wound-wait scheme transaction T_i is wound and rollback because T_j requests a data item that it holds and $TS(T_i) > TS(T_j)$. Now suppose that data item is held by T23 and T22 requests the data item held by T23. Then T23 is wound and rollback because $TS(23) > TS(22)$. When T23 is again restarted and requests the data item T22 then again, $TS(23) > TS(22)$ and T23 will wait for transaction T22 and T23 will not rollback again. Thus, there may be few rollbacks in the wound wait scheme.

Timeout-Based Schemes

Another simple approach to deadlock handling is based on lock timeouts. In this approach, a transaction that has requested a lock waits for at most a specified amount of time. If the lock has not been granted within that time, the transaction is said to time out, and it rolls itself back and restarts. If there was in fact a deadlock, one or more transactions involved in the deadlock will time out and roll back, allowing the others to proceed. This scheme falls somewhere between deadlock prevention, where a deadlock will never occur and deadlock detection and recovery.

Uses of Timeout-Based Schemes

The timeout scheme is particularly easy to implement, and works well if transactions are short, and if long waits are likely to be due to deadlocks.

Limitations

- It is hard to decide how long a transaction must wait before timing out. Too long a wait results in unnecessary delays once a deadlock has occurred. Too short a wait results in transaction rollback even when there is no deadlock, leading to wasted resources.
- Starvation is also a possibility with this scheme.

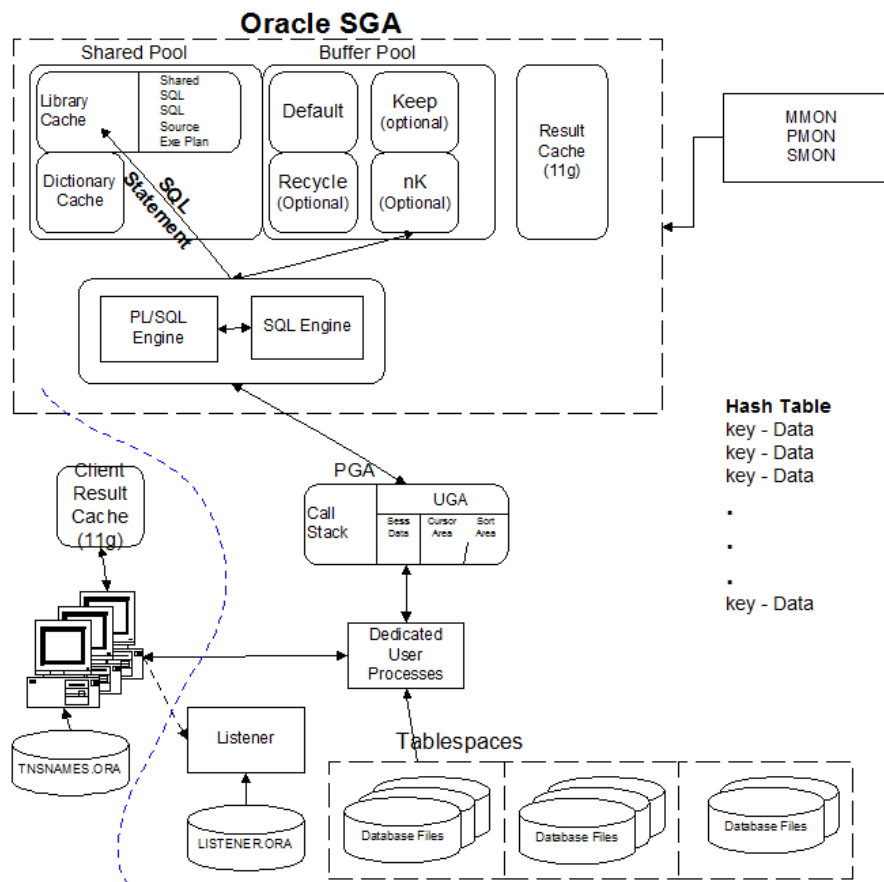
Hence the timeout-based scheme has limited applicability.

9. Explain the various background process of Oracle in detail. (April 2015)

Oracle works is fundamental to tuning SQL statements. This provides an overview of important concepts in Oracle architecture and how they can impact SQL performance tuning. All users have to connect or login to Oracle. The listener is waiting on a predetermined port and 'listens' for traffic from the network. When a request is made to login, the listener process verifies the login credentials and if they pass, a dedicated user process (or if MTS...is assigned to the pool of available shared user processes) is started and a Program Global Area (PGA) is established. This PGA contains an area for any sorts the user will request as well as a cursor area for each SQL submitted. This cursor area is where the

rows/result set from Oracle will be held while negotiating with the application on how to return the rows/result set.

Oracle Architecture



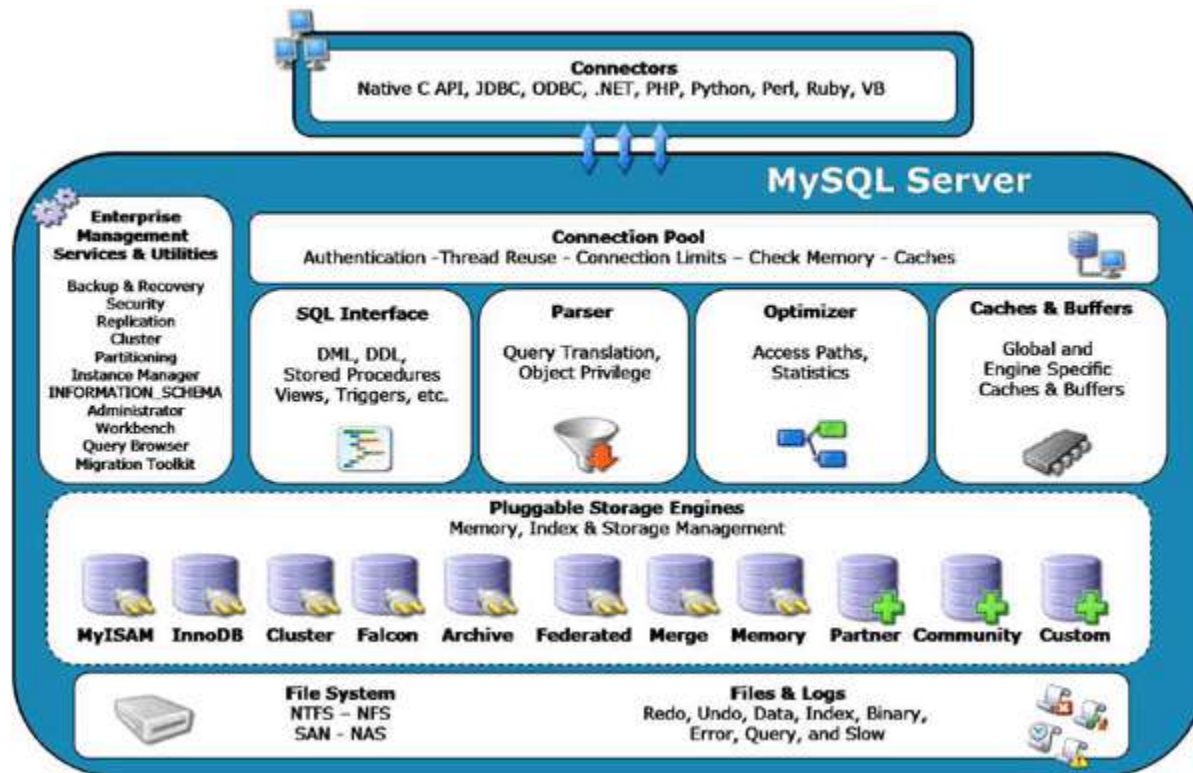
The SQL is then passed to the library cache. Oracle performs a check sum on the SQL and arrives at a hash value from this check sum. This hash value is used to access a predetermined location in the library cache. Oracle checks to see if the SQL in that slot (if there is SQL in the slot) matches the SQL being submitted.

If it is a match, then the existing explain plan is used. This is known as a 'Soft Parse.'

10. With a neat diagram discuss about architecture of MySQL in detail. (April 2015)

MySQL Server is a relational database management system developed by Microsoft. As a database, it is a software product whose primary function is to store and retrieve data as requested by other software applications, be it those on the same computer or those running on another computer across a network (including the Internet). There are at least a dozen different editions of Microsoft SQL Server aimed at different audiences and for workloads ranging from small single-machine applications to large Internet-facing applications with many concurrent users. Its primary query languages are T-SQL and ANSI SQL.

Architecture (6 Marks)



The protocol layer implements the external interface to SQL Server. All operations that can be invoked on SQL Server are communicated to it via a Microsoft-defined format, called Tabular Data Stream (TDS). TDS is an application layer protocol, used to transfer data between a database server and a client. Initially designed and developed by Sybase Inc. for their Sybase SQL Server relational database engine in 1984, and later by Microsoft in Microsoft SQL Server, TDS packets can be encased in other physical transport dependent protocols, including TCP/IP, Named pipes, and Shared memory. Consequently, access to SQL Server is available over these protocols. In addition, the SQL Server API is also exposed over web services.

11. Discuss about Database Recovery procedure in detail. (April 2015)(NOV 2015)[April 2017]

Recovery system

Recovering a system from failure crash is called recovery system or crash recovery.

Failure Classification: (6 Marks)

Various types of failure that may occur in a system

1. Transaction failure

A. Logical errors - The Transaction cannot complete due to some internal error condition

B. System errors-The database system must terminate an active transaction due to an error condition. (e.g., deadlock)

2. System crash:

A power failure or other hardware or software failure causes the system to crash.

Fail stop assumption

A non-volatile storage contents are assumed to not be corrupted by system crash. Database systems have numerous integrity checks to prevent corruption of disk data

3. Disk failure

A head crash or similar disk failure destroys all or part of Disk. Destruction is assumed to be detectable: disk drives use checksums to detect failures

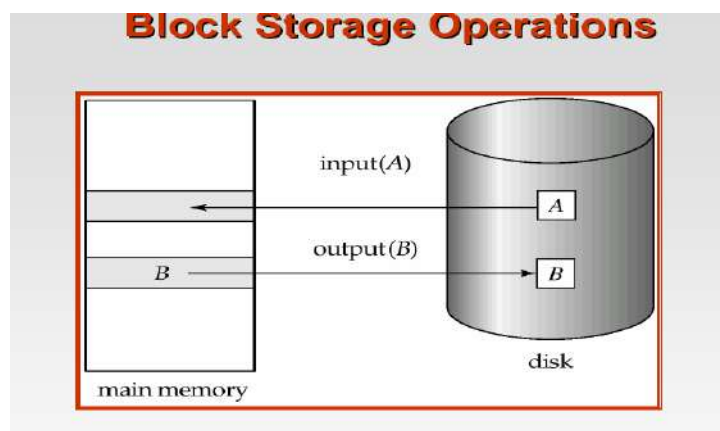
4. Recovery Algorithms

Recovery algorithms are techniques to ensure database consistency and transaction atomicity and durability despite failures.

Recovery algorithms have two parts

5. Actions taken during normal transaction processing to ensure enough information exists to recover from failures
6. Actions taken after a failure to recover the database contents to a state that ensures atomicity, consistency and durability.

(5Marks)



12. Discuss in detail about Concurrency Control. (DEC2018)(MAY 2019)

TIMESTAMP BASED PROTOCOL:

- Timestamp based protocol is the locking protocols and the order between every pair of conflicting transactions at execution time by the first that both members of the pairs request that involves incompatible modes.
- Another method for determining the serializability order is to select an ordering among transaction in advance. The most common method for doing so is to use a timestamp ordering scheme.

TIMESTAMPS:

- This timestamp is assigned by the database system before the transaction T_i starts execution .
- If a transaction T_i has been assigned timestamp $TS(T_i)$ and a new transaction T_j enters the system, then $TS(T_i) < TS(T_j)$. There are two simple method for implementing this scheme.
 1. Use the value of the *system clock* as the timestamp, (i.e) a transactions timestamp is equal to the value of the clock when the transaction enters the system.
 2. Use a logical counter that is incremented after a new timestamp has been assigned and a transaction timestamp is equal to the value of the counter when the transaction enter the system.

To implement this scheme we associate with each data item Q two timestamp values.

- W-timestamp(Q) denotes the largest timestamp of any transaction that executed write(Q) successfully.
- R-timestamp(Q) denotes the largest timestamp of any transaction that executed read(Q) successfully.

THE TIMESTAMP-ORDERING PROTOCOL:

The timestamp-ordering protocol ensures that any conflicting read and write operations are executed in timestamp order. This protocols operates as follows:

1. Suppose that transaction T_i issues read(Q).

- If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i needs to read a value of Q that was already overwritten. hence the read operation is rejected and T_i is rolled back.
- If $TS(T_i) \geq W\text{-timestamp}(Q)$ then the read operation is executed and $R\text{-timestamp}(Q)$ is set to the maximum of $R\text{-timestamp}(Q)$ and $TS(T_i)$.

2. Suppose that transaction T_i issues $write(Q)$.

- If $TS(T_i) < R\text{-timestamp}(Q)$ the the value of Q that T_i is producing was needed previously and the system assumed that the value would never be produced.
- If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of Q . hence the system rejects this write operation and rolls T_i back.
- Otherwise the system executes the write operation and sets $W\text{-timestamp}(Q)$ to $TS(T_i)$.

The protocol can generate schedules that are not recoverable. However it can be extended to make the schedules recoverable in one several ways

- Recoverability and cascadelessness can be ensured by performing all writes together at the end of the transaction.
- Recoverability and cascadelessness can also be guaranteed by using a limited form of locking whereby reads of uncommitted items are postponed until the transaction the updated the item commits.
- Recoverability alone can be ensured by tracking uncommitted writes and allowing a transaction T_i to commit only after the commit of any transaction that wrote a value that T_i read.

Example: The following schedule is possible under the timestamp ordering protocol. Since $TS(T_{14}) < TS(T_{15})$, the schedule must be conflict equivalent to schedule $\langle T_{14}, T_{15} \rangle$

read(B)

read(B)

$B := B - 50$

write(B)

read(A)

read(A)

display(A+B)

$A := A + 50$

write(A)

display(A+B)

T14 T15

Thomas' Write rule:

- Let us consider schedule 4 and apply the timestamp ordering protocol. Since T16 starts before T17 we shall assume that $TS(T16) < TS(T17)$.
- The read(Q) operation of T16 succeeds as does the write (Q) operation, we find that $TS(T16) < W\text{-timestamp}(Q)$, since $W\text{-timestamp}(Q) = TS(T17)$.
- Thus the write (Q) by T16 is rejected and transaction T16 must be rolled back.
- Although the rollback of T16 is required by the timestamp ordering protocol it is unnecessary. Since T17 has already written Q, the value that T16 is attempting to write is one that will never need to be read.
- Any transaction T_i with $TS(T_i) < TS(T17)$ that attempts a read(Q) will be rolled back. Since $TS(T_i) < W\text{-timestamp}(Q)$.

The modification to the timestamp-ordering protocol called Thomas write rule is this. Suppose that transaction T_i issues write(Q).

1. If $TS(T_i) < R\text{-timestamp}(Q)$, then the value of Q that T_i is producing was previously needed and it had been assumed that the value would never be produced. Hence the system rejects the write operation and rolls T_i back.
2. If $TS(T_i) < W\text{-timestamp}(Q)$ then T_i is attempting to write an obsolete value of Q. Hence the write operation can be ignored.
3. Otherwise the system executes the write operation and sets $W\text{-timestamp}(Q)$ to $TS(T_i)$.

Under Thomas' write rule the write(Q) operation of T16 would be ignored. The result is a schedule that is view equivalent to the serial schedule $\langle T16, T17 \rangle$.

13. Explain Concurrency in Index Structures (DEC 2018) (MAY 2019)**

It is possible to treat access to index structures like any other database structure, and to apply the concurrency-control techniques discussed earlier. However, since indices are accessed frequently, they would become a point of great lock contention, leading to a low degree of concurrency. Luckily, indices do not have to be treated like other database structures. It is perfectly acceptable for a transaction to perform a lookup on an index twice, and to find that the structure of the index has changed in between, as long as the index lookup returns the correct set of tuples. Thus, it is acceptable to have nonserializable concurrent access to an index, as long as the accuracy of the index is maintained.

We outline two techniques for managing concurrent access to B+-trees. The bibliographical notes reference other techniques for B+-trees, as well as techniques for other index structures.

The techniques that we present for concurrency control on B+-trees are based on locking, but neither two-phase locking nor the tree protocol is employed. The algorithms for lookup, insertion, and deletion are those used in Chapter 12, with only minor modifications.

The first technique is called the crabbing protocol:

- When searching for a key value, the crabbing protocol first locks the root node in shared mode. When traversing down the tree, it acquires a shared lock on the child node to be traversed further. After acquiring the lock on the child node, it releases the lock on the parent node. It repeats this process until it reaches a leaf node.

- When inserting or deleting a key value, the crabbing protocol takes these actions:

It follows the same protocol as for searching until it reaches the desired leaf node. Up to this point, it obtains only shared locks.

It locks the leaf node in exclusive mode and inserts or deletes the key value.

If it needs to split a node or coalesce it with its siblings, or redistribute key values between siblings, the crabbing protocol locks the parent of the node in exclusive mode. After performing these actions, it releases the locks on the node and siblings.

If the parent requires splitting, coalescing, or redistribution of key values, the protocol retains the lock on the parent, and splitting, coalescing, or redistribution propagates further in the same manner. Otherwise, it releases the lock on the parent.

The protocol gets its name from the way in which crabs advance by moving side-ways, moving the legs on one side, then the legs on the other, and so on alternately. The progress of locking while the protocol both goes down the tree and goes back up (in case of splits, coalescing, or redistribution) proceeds in a similar crab-like manner.

Once a particular operation releases a lock on a node, other operations can access that node. There is a possibility of deadlocks between search operations coming down the tree, and splits, coalescing or

redistribution propagating up the tree. The system can easily handle such deadlocks by restarting the search operation from the root, after releasing the locks held by the operation.

The second technique achieves even more concurrency, avoiding even holding the lock on one node while acquiring the lock on another node, by using a modified version of B+-trees called **B-link trees**; B-link trees require that every node (including internal nodes, not just the leaves) maintain a pointer to its right sibling. This pointer is required because a lookup that occurs while a node is being split may have to search not only that node but also that node's right sibling (if one exists). We shall illustrate this technique with an example later, but we first present the modified procedures of the **B-link-tree locking protocol**.

- **Lookup.** Each node of the B+-tree must be locked in shared mode before it is accessed. A lock on a nonleaf node is released before any lock on any other node in the B+-tree is requested. If a split occurs concurrently with a lookup, the desired search-key value may no longer appear within the range of values represented by a node accessed during lookup. In such a case, the search-key value is in the range represented by a sibling node, which the system locates by following the pointer to the right sibling. However, the system locks leaf nodes following the two-phase locking protocol, as Section 16.7.3 describes, to avoid the phantom phenomenon.

- **Insertion and deletion.** The system follows the rules for lookup to locate the leaf node into which it will make the insertion or deletion. It upgrades the shared-mode lock on this node to exclusive mode, and performs the insertion or deletion. It locks leaf nodes affected by insertion or deletion following the two-phase locking protocol, as Section 16.7.3 describes, to avoid the phantom phenomenon.

- **Split.** If the transaction splits a node, it creates a new node according to the algorithm of Section 12.3 and makes it the right sibling of the original node. The right-sibling pointers of both the original node and the new node are set. Following this, the transaction releases the exclusive lock on the original node and requests an exclusive lock on the parent, so that it can insert a pointer to the new node.

UNIVERSITY QUESTIONS

1. Explain deadlock handling in detail (Nov 2012)(April 2018) **(MAY 2019)** (Question No. 8)
2. Explain about Microsoft SQL server in detail (April 2012) (Nov 2014) (Question No. 5)
3. Explain various models of locking a data item. Also explain two-phase locking protocol (Question No. 7)
4. Explain the various background process of Oracle in detail. (April 2015) (Question No. 9)
5. With a neat diagram discuss about architecture of MySQL in detail. (April 2015) (Question No. 10)
6. Discuss about Database Recovery procedure in detail. (April 2015)(NOV 2015) [April 2017](Question No. 11)
7. State and explain the Two-phase locking protocol with an example. (NOV 2015) [April 2017] (April 2018) (Question No. 3)
8. Outline the drawbacks of the no-steal and force buffer management policies. (NOV 2017) (April 2018)
9. Explain the following:
 - (a) Recovery Algorithm
 - (b) Buffer Algorithm (April 2018)
10. Designate the Concurrency in Index Structure. **(DEC2018)** (MAY 2019)(Question No. 13)
11. Explain IBM DB2 in detail. **(DEC2018)**(Question No. 13)